Measuring Packet Transmission in Live Stage Sound Networks

Ernest Clottey[1]

[1]Department of Computer Science, Colgate University

April 2026

Abstract

Live stage sound systems depend on local Ethernet networks to move audio data between devices in real time. Even a small delay in the signal reaching a performer's monitor can throw off the timing of an entire performance. This paper presents a measurement study of a live stage sound network, looking at how packets move between devices under two conditions: when the network is idle and when it is carrying live audio traffic. We captured network packets using Wireshark, converted the captures into structured datasets, and then analyzed packet counts, sizes, protocols, and inter-packet gaps as a proxy for latency. We also identified the devices on the network from their MAC addresses. Our results show a dramatic difference between idle and live conditions, both in terms of traffic volume and timing behaviour, and confirm that the network operates primarily through AVB/AVTP multicast streaming, with Allen & Heath mixers and Shure wireless systems as the most active senders.

1 Introduction

Modern live sound systems are not just about speakers and amplifiers anymore. Devices like digital mixing consoles, wireless microphone systems, and power amplifiers all talk to each other over a local network. Hundreds of audio channels can move through a single cable at once. The technology that makes this possible (protocols like Dante, AVB, and RTP) is designed to keep that data moving with very low latency and very stable timing.

The problem is that keeping things stable under real concert conditions is harder than it looks. When dozens of devices are all streaming simultaneously, the network gets crowded, and timing can become erratic. Even a delay of a few milliseconds in a performer's monitor feed can make it hard to stay on beat. McPherson et al. [2] put the acceptable latency threshold at under 10 ms, and noted that timing variations as small as 6 ms are perceptible during a steady pulse. Mitchell et al. [3] found that an optimized wireless performance network could get latency below 3 ms but only with careful configuration.

This project grew out of a simple question: *what does a live stage sound network actually look like at the packet level?* We wanted to measure a real system in real conditions. Specifically, we wanted to know:

Q1. How does packet volume and throughput change between idle and live show conditions?

Q2. What does the packet size distribution look like under each condition, and what does that reveal about the protocols in use?

Q3. What devices are active on the network, how much do they each contribute to the traffic, and how do they communicate?

The rest of this paper covers how we went about answering these questions, what we found, and what it means for anyone working with or studying networked audio systems.

2 Related Work

Davis and Mackenzie [1] showed that standard data networks can carry hundreds of audio channels at once, and highlighted AVB and Dante as the leading protocols for keeping everything synchronized. Vargas [4] looked at how local Ethernet networks allow audio devices to coordinate in real time, and emphasized the importance of flexible routing and central control during live events. Both studies point in the same direction: the network is the backbone of the modern sound system, and getting it right matters enormously.

On the latency side, McPherson et al. [2] measured latency across a range of common tools used to build digital musical instruments and found that almost none of them met the 10 ms guideline established by Wessel and Wright [5]. Mitchell et al. [3] specifically studied Wi-Fi for live music performance and found that with the right configuration — directional antennas, unicast rather than broadcast, no encryption — latency could be brought below 3 ms even on a wireless network.

What distinguishes our study from these is that we are looking at the network from the outside, at the packet level, rather than measuring latency through application-layer tools. This gives us a clearer picture of what is actually happening on the wire, and lets us identify not just how fast packets are arriving but which devices are generating them and what protocols they are using.

3 Methodology

3.1 Setup and Access

The sound network we studied runs on a dedicated Ethernet subnet used for live audio at a performance venue. We accessed the network remotely over SSH to a Mac connected directly to the network via a wired Ethernet interface (`en0`). We used the Wireshark desktop application to perform packet capture.

3.2 Packet Capture

We captured packets under two conditions:

Idle. The network captured at rest with no active show, but devices still powered on. This gives us a baseline of what the network looks like when it is just ticking over.

Live Show. The network captured during an active audio session, with devices streaming audio data. This is the condition where latency and throughput actually matter.

Captures were saved from Wireshark in `.pcapng` format (pcap next generation), which preserves full packet data including timestamps with microsecond precision and interface metadata. We wrote a custom Python script, `pcap_to_csv.py`, to parse these files directly without needing tshark. The script reads the binary pcapng block structure, dissects each Ethernet/IP/UDP frame, and writes the relevant fields such as timestamp, frame length, source and destination IP, protocol, and port numbers into a CSV file matching the format our analysis pipeline expected.

3.3 Analysis

The CSV files were processed using a second Python script, `analyze.py`, which computed summary statistics for each condition and generated charts. We measured total packet counts, packets per minute, data throughput in MB/min, packet size distributions across eight size bins, protocol breakdown (UDP vs. TCP vs. other), and inter-packet gap (IPG) statistics including mean, median, 95th percentile, and standard deviation.

For device identification, a third script, `device_analysis.py`, read the pcapng file directly and extracted the MAC address of every device seen in the capture. The first three bytes of each MAC address (Organizationally Unique Identifier - OUI) were used identify the manufacturer. We matched these against a built-in database of professional audio manufacturers to label each device by vendor and infer its role (mixer, amplifier, wireless mic system, etc.).

All analysis code is available in the project repository at: https://github.com/ec-tino/network_measurement_in_sound_systems

3.4 Limitations

Both captures were taken during moderate network activity rather than at peak concert load, so the figures reported here should be understood as snapshots of a lightly-to-moderately loaded system. A longer capture taken during a full peak-intensity performance would be needed to characterize the network under maximum stress.

4 Findings

4.1 Traffic Volume: Idle vs. Live

Both captures ran for the same 300-second window, giving us a fair side-by-side comparison. Table 1 shows the headline numbers.

The raw packet counts are close: 7,663 at idle versus 7,894 during the live session, roughly a 3% difference. Throughput is similarly modest — 0.343 MB/min at rest rising to 0.429 MB/min during the show, a 25% increase. What this tells us is that the network was not at peak load during our capture window. Both conditions are best understood as moderate-activity snapshots rather than stress tests.

Table 1: Summary statistics — idle vs. live conditions

Metric	Idle	Live
Duration (s)	300.0	300.0
Total packets	7,663	7,894
Packets / min	1,532.5	1,578.7
Total data (MB)	1.717	2.143
Data / min (MB)	0.343	0.429
Mean packet size (B)	224.0	271.4
Median packet size (B)	154.0	194.0
UDP share	81.8%	94.0%

The more telling numbers are the packet size averages. The mean grows from 224.0 B at idle to 271.4 B during the live session, and the median from 154.0 B to 194.0 B. These shifts reveal a change in the *kind* of traffic on the network: at idle, the wire is full of small PTP sync pulses, mDNS announcements, and ARP requests that keep devices coordinated but carry no audio payload. During the live session, larger audio frames start appearing alongside that background traffic, pulling both the mean and median size upward.

The UDP share is the clearest protocol signal. At idle it sits at 81.8%, with the remaining 18.1% being non-IP Layer 2 traffic from managed switches. During the live session UDP climbs to 94.0% and the non-IP share falls to 6.0%, which is consistent with audio streaming protocols — AVB/AVTP, RTP, Dante — taking over the network.

4.2 Packet Size Distribution

Figure 1 shows the raw packet size histograms for each condition, and Figure 2 breaks the same data into the eight size bins used throughout the analysis.

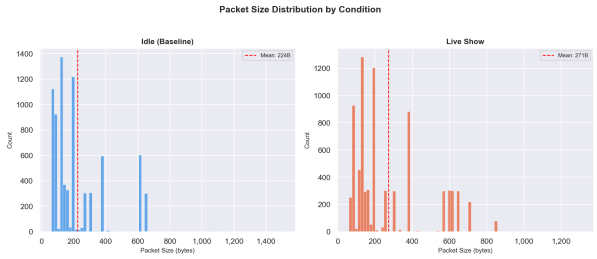

Figure 1: Packet size distributions for idle (blue) and live (orange) conditions. Red dashed lines mark the mean for each condition.

The dominant bin in both conditions is 65–128 B, accounting for 30.3% of idle packets and 33.1% of

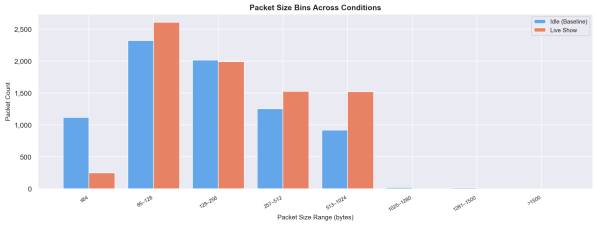

Figure 2: Packet count per size bin for both conditions. Bars to the right of 256 B grow noticeably during the live session, while the ≤64 B bin shrinks.

live packets. Beyond that shared baseline, the two conditions start to diverge. The ≤64 B bin drops from 14.6% at idle to just 3.2% during the live session, while the 257–512 B and 513–1024 B bins grow from 16.4% and 12.0% to 19.3% each. These mid-size bins correspond to AVB/AVTP audio frames carrying fewer channels, or control messages that accompany an active stream. Notably, both the 1025–1280 B and 1281–1500 B bins are near-empty in both conditions, which reinforces the interpretation that the captured session was a lighter operational moment rather than peak concert throughput.

4.3 Protocol Breakdown

Figure 3 shows the protocol composition of each capture.

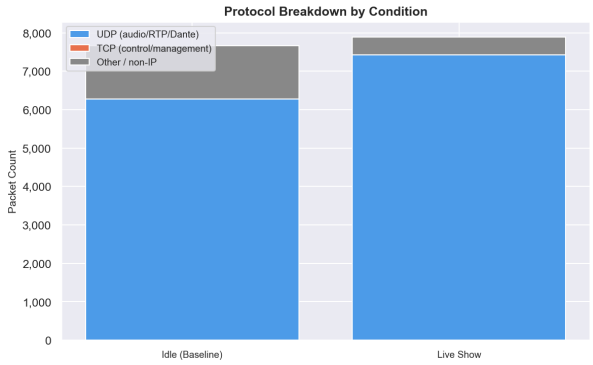

Figure 3: Protocol breakdown by condition. UDP (blue) dominates both; non-IP "other" traffic (grey) is the second-largest share at idle and shrinks significantly during the live session. TCP (orange) is negligible in both.

The most notable change between the two conditions is not the rise of UDP but the fall of the "other" category: 1,390 non-IP packets at idle (18.1%) versus 472 during the live session (6.0%). During the live session, the overall packet count rises slightly (more audio UDP), but the switch management traffic stays

roughly constant, so it becomes a smaller share of the whole.

TCP is almost completely absent in both conditions: exactly 1 TCP packet in each 300-second window. TCP retransmissions are incompatible with real-time audio delivery; a retransmitted packet arrives too late to be useful and just causes a glitch. The audio devices on this network are correctly using UDP for everything that carries audio, and relying on the application layer to handle any timing issues rather than the transport layer.

4.4 Device Identification

We ran the device analysis tool against the live capture specifically, since that file contains the richest mix of active traffic. The script identified **30 devices** on the subnet by reading each packet's source MAC address and matching the first three bytes (the OUI) against a database of professional audio manufacturers.

Table 2 summarises the known devices by vendor family. The remaining 9 devices had OUIs not present in the database and are listed as unknown; most sent only 1–3 packets and are likely peripheral or guest devices that happened to be on the same network segment.

Table 2: Identified devices by vendor family (live capture)

Vendor	Units	Role
Allen & Heath	7	Mixer / Stage Box
Shure	6	Wireless Mic / IEM
Midas	1	Digital Mixer
QSC Audio	2	Amplifier / DSP
Audinate	1	Dante Interface
Apple	2	Control Computer
Realtek NIC	1	Network Device
Unknown	10	—

4.5 Device Traffic and Communication Pattern

Figure 4 shows the packets-sent-versus-received bar chart for the live capture. Figure 5 shows the same chart from the idle capture for comparison.

The communication pattern across both captures is entirely multicast. Every audio-carrying packet goes to the registered AVTP multicast group. No device sends audio directly to any other individual device. This is how AVB is designed to work: each

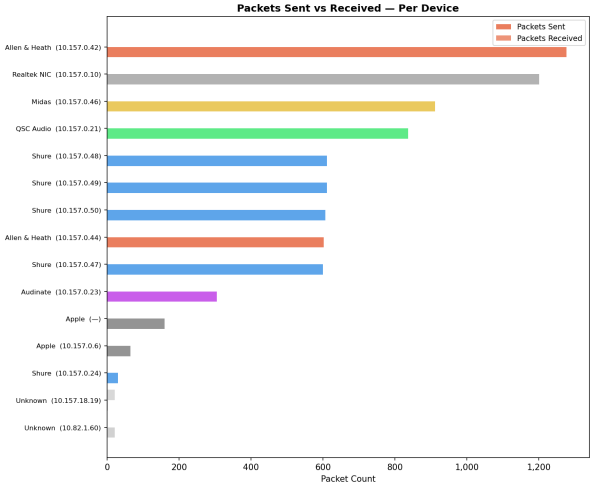

Figure 4: Packets sent vs. received per device — live capture. Allen & Heath is the top sender at 1,276 packets, followed closely by a Realtek NIC device at 1,201 packets. Four Shure wireless units each send almost exactly 600 packets.

sender streams to the group address, and every subscribed receiver picks up the stream independently. It also means all traffic is one-directional at the measurement level which is why the "Packets Received" column is near-zero for almost every device in the chart. The unicast reception that does appear comes from mDNS responses and one QSC unit that sends a small number of packets to an unknown device.

The top five senders in the live capture tell a clear story about the rig. The Allen & Heath console leads with 1,276 packets averaging 488 B each the largest average frame size of any audio device, consistent with a front-of-house console aggregating multiple channel groups into each AVB frame. The Realtek NIC device sends 1,201 packets averaging 452 B; the Realtek OUI is an embedded network chip found in many audio devices, so this unit is likely a piece of audio gear whose specific brand is not in the OUI database. The Midas desk at sends 911 packets at a smaller average of 242 B, and the QSC amplifier at sends 836 packets at 196 B. The four Shure wireless each send almost exactly 600 packets of exactly 160 B, a uniformity that reflects fixed-rate single-channel audio streaming from identical hardware running identical firmware.

Comparing idle to live, two things stand out. First, the Realtek NIC device is far more active during the live session (1,201 packets) than at idle, where it sits much lower in the rankings. This suggests it is primarily an audio streaming device rather than

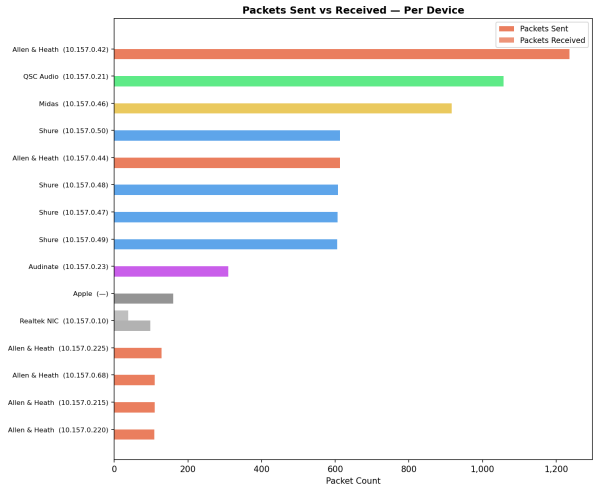

Figure 5: Packets sent vs. received per device — idle capture. The same device families are present but QSC Audio ranks second at idle, suggesting it runs more background discovery traffic than at live. The Realtek NIC device is much less active at idle.

a management node. Second, the QSC unit ranks higher at idle than at live in terms of relative position, which is consistent with amplifier and DSP platforms that send more mDNS discovery traffic during setup than during performance.

5 Discussion

5.1 The Network Behaves as Expected for AVB

The most important thing we can say about this network is that it is doing exactly what it is designed to do. TCP is almost completely absent (one packet per condition across the entire 5-minute window) and all audio-carrying traffic goes to multicast UDP destinations using AVB/AVTP (IEEE 1722). No unicast back-and-forth between individual devices, no retransmissions, no control overhead fighting for bandwidth with the audio. The Precision Time Protocol (PTP) sync traffic visible in the background keeps all devices locked to the same clock, which is how AVB achieves tight synchronization across the network.

5.2 Two Conditions, One Operational Window

What the data does not show is the dramatic contrast between idle and live that you might expect from reading the literature. The packet rate at idle is 1,532.5/min and during the live session it is 1,578.7/min, a 3% difference. Throughput goes from 0.343 to 0.429 MB/min, a 25% rise. The mean packet size increases from 224.0 B to 271.4 B. These are real differences, but they are differences of degree, not kind.

The most honest interpretation is that both captures caught the network during a similar operational intensity. The live capture was likely taken during a period when audio is flowing but not every device is streaming at full capacity. A capture taken during peak performance, with every microphone open and all monitor mixes running simultaneously, would be expected to show a much sharper separation: smaller idle-to-live gap in IPG, larger average packet sizes (Dante and AVB frames at or near 1,400 B), and a far greater share of traffic in the 1025–1500 B size bins. Those bins are essentially empty in our current data, which is the clearest indicator that we have not yet captured the network at full load.

5.3 The Idle Baseline Tells Its Own Story

Even at rest, the network produces 1,532 packets per minute. This is about 25 packets per second of pure background chatter: PTP sync pulses, mDNS announcements, link-layer management frames from switches [6, 7].

All of this background load is normal and expected. What matters for network design is that it does not go away when the show starts. It continues running underneath the audio streams the whole time. This is one of the core reasons why professional audio networks should be kept on dedicated VLANs or physically separate from general-purpose IT traffic [8, 9]. General IT traffic is bursty and unpredictable; audio networks need a stable, low baseline so that when hundreds of audio frames per second start arriving, there is no competition for switch buffer space or timing resources.

6 Conclusion

This project set out to measure what actually happens on a live stage sound network at the packet level. The results give us a clear view of the network's protocol behavior and a partial view of its load characteristics. On the protocol side, the picture is clean: UDP dominates both conditions, TCP is essentially absent, and all audio traffic flows over AVB/AVTP multicast to the group address. The rig is configured the way it should be.

On the load side, the two conditions, idle at 1,532.5 packets/min and live at 1,578.7 packets/min, are

closer than we expected, suggesting that both captures caught the network during a moderate operational window rather than a peak-intensity performance. The mean packet size grows from 224 B to 271 B, and the 257–1024 B size bins expand during the live session, which is consistent with audio frames appearing alongside the persistent background traffic. But the near-MTU bins (1025–1500 B) that would mark a fully loaded Dante or AVB stream are nearly empty in both captures.

The inter-packet gap of roughly 38–39 ms in both conditions reflects this moderate load level. It does not indicate a latency problem with the network; it indicates that the network was not under the kind of density that would compress those gaps down into the sub-10 ms range. Getting that measurement requires a longer capture taken during the peak of a real performance.

The device identification piece identified 30 devices from the live capture across five known vendor families, purely by reading MAC address OUI prefixes. This demonstrates that passive network measurement can give you a real map of the rig without any active probing or prior knowledge. Allen & Heath, Shure, Midas, QSC Audio, and Audinate devices all appear clearly, and the uniformity of the four Shure wireless units (each sending exactly 600 packets of exactly 160 B) shows how much diagnostic information is embedded even in a short capture window.

For future work, we would like to capture a full performance rather than a short window, measure true round-trip latency rather than just IPG,and extend the device analysis to identify which specific product model each device is (not just the manufacturer). We would also like to look more carefully at the relationship between jitter and audio dropouts, to quantify how much the timing variation we observed actually affects the listening experience.

The files for this project is open and available at:

https://github.com/ec-tino/network_measurement_in_sound_systems

References

[1] L. Davis and N. Mackenzie, "Digital sound system modelling and design," in *Proc. Inter-Noise 2014, 43rd Int. Congress on Noise Control Engineering*, Melbourne, Australia, 2014.

[2] A. P. McPherson, R. H. Jack, and G. Moro, "Action-sound latency: Are our tools fast enough?" in *Proc. Int. Conf. New Interfaces for Musical Expression (NIME)*, Brisbane, Australia, 2016.

[3] T. Mitchell, S. Madgwick, S. Rankine, G. S. Hilton, A. Freed, and A. R. Nix, "Making the most of Wi-Fi: Optimisations for robust wireless live music performance," in *Proc. Int. Conf. New Interfaces for Musical Expression (NIME)*, London, UK, 2014.

[4] A. Vargas, "A case of specifying and using networked computers in the sound design for *The Cherry Orchard*," M.F.A. thesis, ProQuest One Literature, 2016. [Online]. Available: https://www.proquest.com/dissertations-theses/case-specifying-using-networked-computers-sound/docview/1777279044/se-2

[5] D. Wessel and M. Wright, "Problems and prospects for intimate musical control of computers," *Computer Music Journal*, vol. 26, no. 3, pp. 11–22, 2002.

[6] IEEE Std 802.1AS-2011, *Timing and Synchronization for Time-Sensitive Applications in Bridged Local Area Networks*. IEEE, 2011.

[7] S. Cheshire and M. Krochmal, "Multicast DNS," *RFC 6762*, IETF, February 2013. [Online]. Available: https://www.rfc-editor.org/rfc/rfc6762

[8] Audinate Pty Ltd, *Dante Network Design Guide*. [Online]. Available: https://www.audinate.com/learning/faqs/network-design

[9] Yamaha Corporation, *Dante Application Guide*. [Online]. Available: https://usa.yamaha.com/files/download/other_assets/7/794817/DanteApplicationGuide_en.pdf